



КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

03

Лекция 3
Указатели, ссылки, категории значений

Модель памяти · адреса и арифметика указателей · const и nullptr · new/delete · ошибки памяти и санитайзеры · ссылки · категории значений · приведения

Вспоминаем, прошлую лекцию

```cpp
#include <iostream>

int main() {
    int *ptr;

    int a {10};
    ptr = &a;

    if (( *ptr == 10) && (ptr != nullptr)) {
        switch(*ptr) {
            case 10:
                std::cout << "10\n";
            case 20:
                std::cout << "20\n";
        }
    }
    delete ptr;

    return 0;
}
```



ПЛАН ЛЕКЦИИ

Десять глав сегодняшней лекции

01 Модель памяти процесса

02 Адрес и разыменование

03 Указатели и массивы

04 const-указатели и nullptr

05 Динамическая память new/delete

> **Сквозная идея:** память — источник многих ошибок; указатели, ссылки и категории значений — способ говорить о ней точно.

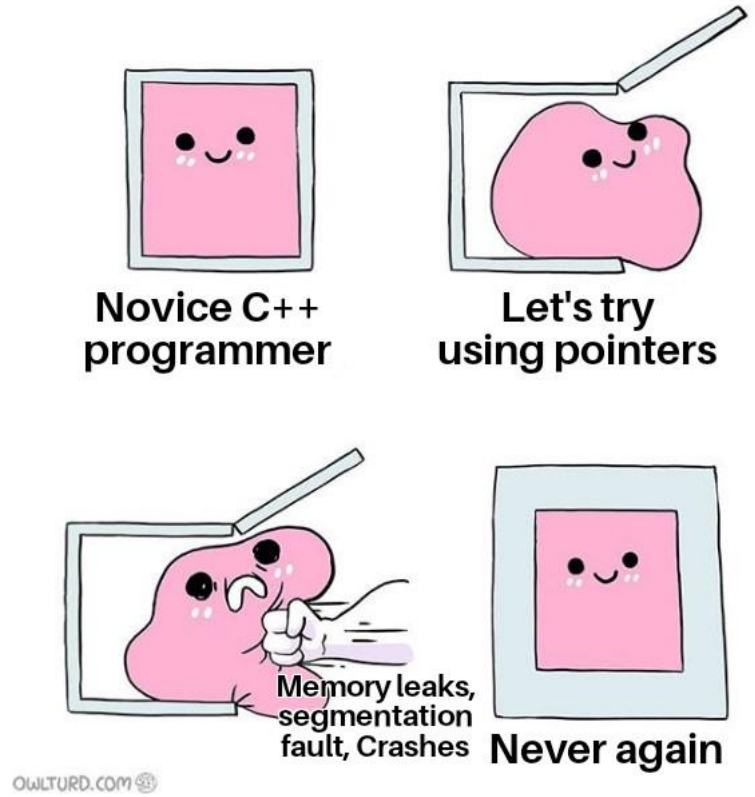
06 Ошибки памяти

07 Санитайзеры

08 lvalue-ссылки

09 Категории значений

10 Приведение типов



Стек, куча, статическая память, код




```
int global_var = 10;                    // статическая
void foo() {
        int local = 20;                  // стек
        int* h = new int(40); delete h;  // куча
}
```

- **Стек** — быстро, LIFO, размер ~1–8 МБ
- **Куча** — гибко, но вручную: утечки, double free, dangling
- **Статическая** — время жизни = программы
- предпочитайте объекты на стеке — RAII-деструктор освободит сам

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/01_memory_model

&x, *ptr и арифметика указателей

```
int x = 42;
int* ptr = &x;                  // адрес x
*ptr = 100;                     // меняем x

sizeof(int*)                    // 8 на 64-bit — от типа не зависит

int arr[] = {10, 20, 30, 40, 50};
int* p = arr;
*(p + 1)                        // 20 — сдвиг на 1 элемент
p += 3;          *p             // 40
ptrdiff_t d = (arr + 5) - arr;                  // 5
```

- &x — адрес, *ptr — разыменование
- p + n — сдвиг на n элементов (sizeof(*p) байт)
- арифметика — только в пределах одного массива, иначе UB
- разность указателей — std::ptrdiff_t

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/02_address_deref

Имя массива «распадается» в указатель

```
int arr[5] = {1, 2, 3, 4, 5};
arr[2]                  // 3
*(arr + 2)              // 3
2[arr]                  // 3 — a[b] == *(a+b)

void print(int* data, size_t size);                    // размер теряется
```

- имя массива — указатель на первый элемент (decay)
- при передаче в функцию размер теряется — передавайте его отдельно

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/03_pointers_arrays

Четыре комбинации const

```
int* p1;                    // можно менять и значение, и адрес
const int* p2;              // нельзя менять значение (p2 = &y — ок)
int* const p3 = &x; // нельзя перенаправить (*p3 = 5 — ок)
const int* const p4; // ничего нельзя

// Мнемоника: читайте справа налево
// const int*          o  указатель на const int
// int* const          o  const указатель на int

int* ptr = nullptr;             // типобезопасный нулевой указатель
// NULL/0 — неоднозначны при перегрузке: foo(nullptr)
```

- по умолчанию добавляйте const— документирует намерение
- nullptrимеет тип std::nullptr_t
- NULLможет быть 0(int) — ломает перегрузку

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/04_const_pointers_nullptr

Парные операторы — строго по парам

```
int* p = new int(42); delete p;          // один объект
                                           // delete

int* arr = new int[5]{...}; // массив
delete[] arr;                                 // delete[]

// Смешивать delete и delete[] — UB!
int* bad = new int[10];
// delete bad;              // UB! нужно delete[]

int* z = new int{};                          // value-init → 0
```

- new/deleteи new[]/delete[]— строго парные
- new int{}— обнуление
- в современном C++ — минимизируйте: vector, умные указатели

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/05_dynamic_memory

Dangling, leak, double free

```
// Dangling pointer
int* f() { int x = 42; return &x; }                    // x умерла

// Memory leak
void leak() { int* p = new int(42); } // забыли delete

// Double free
int* p = new int(42); int* q = p;
delete p;
// delete q;              // UB — два владельца
```

- после delete— обнуляйте p = nullptr
- ранний return— частая утечка
- общий корень — ручное владение; лечится RAII и умными указателями

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/06_memory_errors

Компилятор ловит ошибки в рантайме

```
g++ -fsanitize=address,undefined -g main.cpp
./a.out
ERROR: AddressSanitizer: heap-
        buffer-overflow
    at main.cpp:5          WRITE of size 4
```

- **ASan** — утечки, выход за границы, use-after-free
- **UBSan** — прочие случаи UB
- объединяйте: `-fsanitize=address,undefined` собирайте
- с `-g`, без агрессивной оптимизации инструмент
- разработки, а не продакшена

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/07_sanitizers

Ссылка — алиас существующего объекта

```
int x = 42; int& ref
= x; ref = 100;          // алиас
                           // меняет x

const int& cref = 42;              // продлевает жизнь временного
// int& bad = 42;                  // ошибка: lvalue к rvalue

for (auto& n : nums) n *= 2;                    // модификация
for (const auto& n : nums) cout;                // чтение без копий
```

- обязана быть инициализирована, не переназначается, не может быть nullptr
- const T&принимает и lvalue, и rvalue
- указатель — когда нужны nullptr, переназначение, арифметика

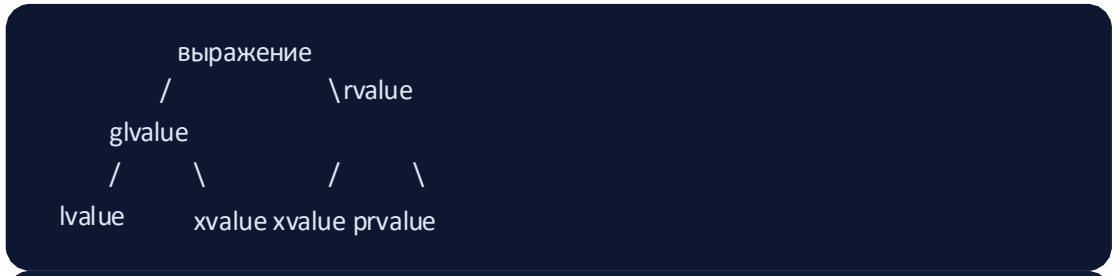
ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/08_references

lvalue, prvalue, xvalue




```
int x = 42; x + 1       // x — lvalue (есть адрес)
std::move(x)            // prvalue (временное)
                        // xvalue (готово к move)
```

Это классификация **выражений** (не типов!) по тому, как они относятся к объектам и перемещению.

lvalue (locator value)

```
lvalue (locator value) int x = 5;
x;              // lvalue — именованная переменная
++x;            // lvalue
arr[0];         // lvalue
*p;             // lvalue (разыменование)
std::cout;      // lvalue
```

Выражение, у которого есть **имя и адрес**, ссылающееся на конкретный объект. Живёт долго, можно взять адрес.

prvalue (pure rvalue)

```
42;                 // prvalue — литерал
x + y;              // prvalue — результат операции
foo();              // prvalue, если возвращает по значению
std::string{};      // prvalue — временный объект
&x;                 // prvalue — адрес это значение
```

Временное значение, у которого нет имени и адреса. Это «чистое» значение — результат вычисления.

xvalue (eXpiring value)

```
std::move(x);                  // xvalue
static_cast<Foo&&>(x);         // xvalue
std::forward<Foo>(x);          // xvalue, если T&&
```

Объект, который **скоро умрёт** — его можно «обобрать» (переместить ресурсы). Обычно результат std::move или приведение к rvalue-ссылке.

Правила

1. Всё, что имеет имя — lvalue. Даже если это rvalue-ссылка

```
void f(Foo&& x) {
    x;                // lvalue! у него есть имя
    std::move(x);     // xvalue — явно превращаем в перемещаемое
}
```

2. Взятие адреса & **требует lvalue** (кроме перегруженного operator&)

3. rvalue-ссылка T&& **привязывается к rvalue (xvalue + prvalue), но не к lvalue**

4. lvalue-ссылка T& **привязывается только к lvalue** (не считая const T&, который привязывается ко всему)

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/09_value_categories

Именованные *_cast вместо C-style

```
int t = static_cast<int>(3.14159);                    // 3

const int* cp = &x;
int* p = const_cast<int*>(cp);                        // снять const
// *p = 100;            // UB, если x изначально const

char* c = reinterpret_cast<char*>(&x); // биты
// C-style (int)pi — слишком мощный, избегать
```

Оператор	Безопасность
static_cast	высокая
const_cast	средняя
reinterpret_cast	низкая
dynamic_cast	высокая (позже)
(T)x	непредсказуемая

ПРИМЕР

Смотрим пример:



https://github.com/DVDemon/OOP_2027_Part_01/tree/main/lecture_03_pointers_references/10_type_casts

ИТОГИ

Ключевые концепции лекции

- **Указатель** хранит адрес; &— адрес, *— разыменование
- **Арифметика указателей** масштабируется на sizeof(*p)
- **nullptr**— типобезопасная замена NULL/0
- new/deleteи new[]/delete[]— парные
- **Ссылка** — алиас: не nullable, не переназначается
- const&продлевает жизнь временного объекта
- **Категории значений:** lvalue, prvalue, xvalue
- **Санитайзеры** (ASan, UBSan) — обязательны при разработке

